

UNIT I: Asymptotic Analysis, Array Addressing, Searching & Sorting Algorithms

25% Exam Weightage

1. Asymptotic Notations

Complexity

Big-O: $f(n) \leq c \cdot g(n)$ • Upper Bound (Worst)Omega (Ω): $f(n) \geq c \cdot g(n)$ • Lower Bound (Best)Theta (Θ): $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ • Tight Bound

- **Order of Growth:** $O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$.
- **Master Theorem:** $T(n) = aT(n/b) + \Theta(n^k \log^p n)$. If $\log_b(a) > k \Rightarrow T(n) = \Theta(n^{\log_b a})$. If equal with $p=0 \Rightarrow \Theta(n^k \log n)$.

2. Multi-D Array Addressing

Exam Must

1D: $\text{Loc}(A[i]) = B + (i - L) \times W$ 2D Row-Major: $B + [(i - L_1) \times N + (j - L_2)] \times W$ 2D Col-Major: $B + [(j - L_2) \times M + (i - L_1)] \times W$

- **B:** Base Address; **W:** Element width in bytes.
- **M, N:** Total rows = $(U_1 - L_1 + 1)$, Total cols = $(U_2 - L_2 + 1)$.
- **3D Row-Major:** $\text{Loc}(A[i][j][k]) = B + [(i - L_1) \times N \times P + (j - L_2) \times P + (k - L_3)] \times W$.

3. Sparse Matrix Storage

Representation

Sparse 4x5

>70% Zeros

Triplet Array: (Row, Col, Val)

Row 0: [TotalRows, TotalCols, NonZeros]

Saves $O(M \times N)$ space $\rightarrow O(\text{NonZero})$

- **Linked List Rep:** Header nodes for each row/col + node triplets (row, col, value, nextRow, nextCol).

4. Master Sorting & Searching Decision Matrix

Comparison

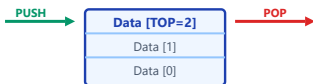
Algorithm	Best Time	Average Time	Worst Time	Space	Stable?	Core Mechanism & Key Exam Fact
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Yes	Sequential scan; works on unsorted lists.
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ iter / $O(\log n)$ rec	N/A	Requires sorted array; mid = low + (high-low)/2 avoids int overflow.
Bubble Sort	$O(n)$ [optimized]	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Repeated adjacent swaps; flag stops early if 0 swaps in pass.
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Finds minimum in unsorted sublist; exactly n-1 swaps guaranteed.
Insertion Sort	$O(n)$ [sorted]	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Inserts key into sorted prefix; optimal for small or nearly sorted arrays.
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$ auxiliary	Yes	Divide & Conquer; guarantees $O(n \log n)$ in all cases; ideal for linked lists.
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$ [skewed]	$O(\log n)$ rec stack	No	Lomuto/Hoare partitioning; worst case if pivot is min/max in sorted array.
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$ auxiliary	No	Builds Max-Heap $O(n)$, repeatedly extracts root $O(\log n)$; in-place.

UNIT II: Stacks, Infix/Postfix/Prefix Expressions, Recursion & Tower of Hanoi

25% Exam Weightage

5. Stack ADT & Operations

LIFO Policy



- **Push:** If top == MAX-1 $\rightarrow$ Overflow. Else top++; stack[top]=val;
- **Pop:** If top == -1 $\rightarrow$ Underflow. Else val=stack[top]; top--;
- **Peek/Top:** stack[top] in $O(1)$ time without removing.
- **Linked Stack:** Top is head; Push = insert_first; Pop = delete_first.

6. Infix, Postfix & Prefix

Shunting-Yard

Precedence: $\wedge$ (R-to-L) > *, / (L-to-R) > +, - (L-to-R)

- **Infix to Postfix Algorithm:**
 - Operand $\rightarrow$ output directly to postfix string.
 - '(' $\rightarrow$ push to operator stack.
 - ')' $\rightarrow$ pop & output until '(' popped.
 - Operator $\rightarrow$ pop higher/equal precedence operators to output, then push incoming operator.
- **Postfix Evaluation Algorithm:** Scan L-to-R. Operand $\rightarrow$ push. Operator $\rightarrow$ pop op2, pop op1, evaluate op1 [op] op2, push result.
- **Prefix Evaluation:** Scan R-to-L. Operand $\rightarrow$ push. Operator $\rightarrow$ pop op1, pop op2, evaluate op1 [op] op2, push result.

7. Recursion & Tower of Hanoi

Stack Memory

Hanoi Moves: $2^n - 1$ • Recurrence: $T(n) = 2T(n-1) + 1$

- **Call Stack Activation Record:** Each recursive call pushes stack frame: [parameters, local vars, return address].
- **Base Case:** Halts recursion; omission causes Stack Overflow.
- **Hanoi Rule (Source S, Aux A, Dest D):**
 1. Move n-1 disks from S to A using D.
 2. Move disk n directly from S to D.
 3. Move n-1 disks from A to D using S.
- **For n=3:** Exactly $2^3 - 1 = 7$ disk movements.

EXAM TRAP 1 (Row vs Column Major): When finding 2D addresses, count elements before target! In row-major, row index determines full rows skipped: $(i - L_1) \times \text{TotalCols}$. In col-major, column index determines full columns skipped: $(j - L_2) \times \text{TotalRows}$. Mixing TotalRows and TotalCols is the #1 lost mark!

EXAM TRAP 2 (Postfix Stack Order): During Postfix Evaluation, the FIRST popped element is operand 2 and the SECOND popped element is operand 1. Evaluation must be op1 / op2 or op1 - op2. Swapping them inverts subtraction/division!

UNIT III: Linear Queues, Circular Queues, Deque, Priority Queue & Linked Lists

25% Exam Weightage

1. Queue ADT & Circular Queue

FIFO Policy

Linear Queue: False Overflow when rear=MAX-1

Circular Enqueue: rear = (rear + 1) % MAX

Circular Dequeue: front = (front + 1) % MAX

- **Circular Full:** (rear + 1) % MAX == front
- **Circular Empty:** front == -1 && rear == -1
- **Deque (Double-Ended):** Insert/delete allowed at both ends. Types: Input-restricted & Output-restricted.
- **Priority Queue:** Elements dequeued by priority, not arrival. Array = O(n) or O(1); Binary Heap = O(log n).

2. Linked List Topologies

Pointers

Type	Node Structure	End Condition
Singly (SLL)	[Data Next*]	last->next == NULL
Doubly (DLL)	[*Prev Data *Next]	next==NULL, prev==NULL
Circular (CLL)	[Data Next*]	last->next == head
Circular DLL	[*Prev Data *Next]	head->prev==last, last->next==head

- **Array vs LL:** Array has O(1) random access but fixed capacity. LL has dynamic sizing, O(1) head insertion, but O(n) sequential search & pointer memory overhead.

3. Pointer Rewiring Formulas

Code Snips

SLL Insert Beg: newNode->next = head; head = newNode;

SLL Delete Beg: temp = head; head = head->next; free(temp);

DLL Insert Mid: newNode->next = cur->next; newNode->prev = cur; cur->next->prev = newNode; cur->next = newNode;

- **Delete Target Ptr in SLL:** Requires traversing with prev pointer to bridge: prev->next = cur->next; free(cur);
- **Header Linked List:** Contains special dummy node at start tracking list metadata (length, max value, count).

UNIT IV: Trees, Binary Search Trees (BST), Graph Representations & BFS/DFS Traversals

25% Exam Weightage

4. Trees & Binary Search Trees

Hierarchical

Max nodes at level i = 2ⁱ • Max in tree = 2^h - 1

BST Property: LeftChild < Root < RightChild

- **Full vs Complete:** Full = every node has 0 or 2 children. Complete = all levels filled except possibly last, filled left-to-right.
- **BST Operations:**
 - Search / Insert: Average O(log n); Worst O(n) if skewed.
 - Delete Case 1 (Leaf): Set parent's pointer to NULL.
 - Delete Case 2 (1 Child): Bypass node to child.
 - Delete Case 3 (2 Children): Replace with *Inorder Successor* (min of right subtree) or *Inorder Predecessor*.

5. Tree Traversals

Recursion

Inorder (L-Root-R): Always yields sorted order for BST

Preorder: Root-L-R • Postorder: L-R-Root



- **Tree Construction Rule:** Inorder + Preorder OR Inorder + Postorder uniquely reconstructs a binary tree. Preorder + Postorder does NOT!
- **Expression Tree:** Internal nodes = operators; Leaves = operands. Inorder traversal = Infix; Postorder = Postfix.

6. Graph Representations & Search

Networks

Max edges in simple graph: V(V-1)/2 (undirected), V(V-1) (dir)

Feature	BFS (Breadth-First)	DFS (Depth-First)
Data Structure	FIFO Queue	LIFO Stack / Recursion
Time Complexity	O(V + E) [Adj List]	O(V + E) [Adj List]
Space	O(V) queue	O(V) call stack
Application	Shortest path (unweighted)	Cycle detection, Topo sort

- **Adjacency Matrix:** O(V²) space; O(1) edge lookup. Dense graphs.
- **Adjacency List:** O(V+E) space; O(degree(u)) edge lookup. Sparse graphs.

7. High-Yield Formula, Recurrence & Exam Checklist

Exam Cram

Concept	Formula / Recurrence	Worst-Case Boundary	Exam Trap / Caution
Tower of Hanoi	T(n) = 2T(n-1) + 1 = 2 ⁿ - 1	O(2 ⁿ) exponential	Remember to state auxiliary peg name explicitly in step 1 and step 3!
Binary Search	T(n) = T(n/2) + O(1) = O(log n)	O(log n)	Array must be strictly sorted before applying binary search.
Merge Sort	T(n) = 2T(n/2) + Θ(n) = Θ(n log n)	Θ(n log n)	Requires O(n) auxiliary memory; cannot be done in-place easily.
Quick Sort	T(n) = T(n-1) + Θ(n) = Θ(n ²) [worst]	O(n ²)	Happens when input is already sorted and first/last element chosen as pivot.
Circular Queue Size	Count = (rear - front + MAX) % MAX + 1	MAX items	Always check front == -1 before applying the distance formula.
Height of BST	h ≥ ⌊log ₂ (n+1)⌋ - 1	h = n-1 (Skewed tree)	Degenerates to singly linked list with O(n) search time if not balanced.
Strictly Binary Tree	Leaf nodes L = Internal nodes I + 1	Total nodes N = 2I + 1	If internal nodes I = 10, then exactly 11 leaves and 21 total nodes.

WRITING ALGORITHMS IN EXAM: Always structure answers as: (1) Formal Definition & Mathematical ADT specification, (2) Step-by-Step Algorithm (checking boundary conditions: Underflow/Overflow/NULL pointer), (3) Trace example with neat diagram, (4) Time & Space Complexity derivation.

TOP 4 EXAM DIAGRAMS: (1) Lomuto Partitioning step-by-step pointers, (2) Circular Queue index wrap-around circle, (3) Singly Linked List deletion pointer rewiring, (4) BST Deletion with Inorder Successor node replacement.